

Ansible_RPI_TouchPlayer

本项目致力于将 Raspberry Pi 4B 打造为一个支持多音源无缝切换的**专业级纯净媒体播放器**。

通过彻底的硬件逻辑重构，本作从传统的单色 OLED 升级为 **3.5 寸全彩 SPI 电容触摸屏**，并基于纯 Python 与 `smbus2` 轮询架构，实现了全屏隐形热区的"盲操"级触控交互。

整个系统架构采用 **Ansible 全自动化部署**，彻底告别繁杂的 Linux 命令行配置。

✨ 核心特性

🎵 多音源无缝路由 (基于 PipeWire)

- 🎵 Squeezelite - 连接 Logitech Media Server，播放无损本地音乐库。
- 📱 AirPlay 2 - 从 iPhone/iPad/Mac 推送系统级音频。
- 🔵 Bluetooth A2DP - 接收任何蓝牙设备的音频流。

🖥️ C + LVGL 高性能触控 UI

- **架构选型思考 (User-space vs. Kernel Overlay)**: 针对微雪 3.5寸 SPI 电容屏，我们坚决**放弃了**将屏幕挂载为系统级 Framebuffer (`/dev/fb0` , 通过 `mipi-dbi-spi` 或 `fbcp` 帧同步) 的硬核方案。
 - **原因**: 系统级 Framebuffer 驱动会在 SPI 总线上以固定帧率进行全局刷新。在 480x320 16-bit 的分辨率下，这会造成极大的 CPU 占用和不可避免的画面撕裂（受限于 SPI 带宽物理极限）。而且由于本设备是无头（Headless）音频播放器，并没有桌面渲染需求。
 - **方案**: 我们采用官方推荐的**纯用户态 (User-space) 直驱模式**。基于 C 语言直接向 `/dev/spidev` 和 `/dev/i2c` 发送指令，结合 LVGL 强大的脏矩形（Dirty Region）渲染机制，实现**局部刷新**。只有当 UI 组件（如音量条）发生变化时，才通过 SPI 发送变化区域的像素，将 CPU 消耗降至最低，同时保障极致的流畅度和性能。
- **原生 LVGL 渲染**: 采用纯 C 与 LVGL 显示栈，目标对接 Waveshare 480x320 电容触摸屏，替换掉旧的 Python / OLED 方案。
- **全屏触控界面**: 为 480x320 屏幕设计明确的触控区域与状态反馈，提升操作直观性与

延迟表现。

- **软 I2C + SPI 硬件隔离：**将触摸控制与屏幕刷新分别通过软件 I2C 与 SPI 传输，保持与 WM8960 音频总线的物理隔离。

🔊 硬件级发声底座

- 采用 **Waveshare WM8960 Sound Board** 提供高保真 I2S 硬件解码输出。
- 独立音量控制与优先级自动管理。

🔌 核心硬件接线指南 (Pinout Guide)

⚠️ 架构警告 (冲突规避)

本项目同时挂载了 SPI 触摸屏与 I2S 音频板。**绝对不能**按照屏幕官方维基的默认方式连接！请严格遵循以下两张接线表，我们已从物理层和软件层彻底解决了总线冲突。

1. WM8960 音频板 (独占硬 I2C-1 与 I2S)

作为核心发声单元，音频板保持独占树莓派的标准音频与控制总线。
(注：保留 38 脚录音数据线是为了满足 Linux ALSA 驱动的全双工初始化自检，防止底层报错)

树莓派主控端 (RPI 4B)	绝对数据流向	WM8960 音频板 (外设端)	功能说明
PIN 2 或 4 (5V)	→ 供电 →	5V	5V 主供电
PIN 6 或 9 (GND)	→ 接地 →	GND	统一接地
PIN 3 (BCM 2)	↔ 双向 ↔	SDA	I2C-1 音频控制数据
PIN 5 (BCM 3)	→ 发送 →	SCL	I2C-1 音频控制时钟
PIN 10 (BCM 1)	→ 发送 →		I2S 位时钟 (超采样时钟)

PIN 12 (BCM 18)	→ 发送 →	CLK	I2S 时钟 (极具敏感性, 切勿被抢占)
PIN 35 (BCM 19)	→ 发送 →	LRCLK (WS)	I2S 左右声道帧时钟
PIN 40 (BCM 21)	→ 发送 →	DAC (RXSDA)	播放: 将树莓派的数字音频推给声卡发声
PIN 38 (BCM 20)	← 接收 ←	ADC (TXSDA)	录音: 声卡麦克风数据发给树莓派 (迎合驱动自检)

2. SPI 彩色电容触摸屏 (SPI0 + 软 I2C-3 + PWM1)

产品资料: [https://www.waveshare.net/wiki/3.5inch_Capacitive_Touch_LCD?hl=zh-CN]

显示采用标准 SPI 推流, 触控采用 GPIO 模拟的独立 I2C-3 总线 (完全物理隔离音频总线), 同时采用**纯轮询模式**放弃中断引脚。

LCD引脚号	LCD引脚	树莓派物理引脚	树莓派 (BCM)	功能说明
1	VCC	PIN 2	5V	屏幕及触控芯片主供电 (5V)
2	3V3	NC	NC	悬空不接 (已使用5V供电)
3	GND	PIN 6	GND	统一接地
4	MISO	PIN 21	BCM 9	SPI0 屏幕数据返回 (可选)
5	MOSI	PIN 19	BCM 10	SPI0 传输像素数据至屏幕
6	SCLK	PIN 23	BCM 11	SPI0 传输时钟
7	SD_CS	NC	NC	SD 卡片选 (未使用)
8	LCD_CS	PIN 24	BCM 8	SPI0 硬件片选
9	LCD_DC	PIN 22	BCM 25	屏幕数据/命令切换引脚
10	LCD_RST	PIN 13	BCM 27	屏幕显示芯片复位

11	LCD_BL	PIN 33	BCM 13	屏幕背光控制 (PWM1, 避开 I2S CLK)
12	TP_SDA	PIN 7	BCM 4	软 I2C-3 触控数据线 (复用原 SSD1306 总线)
13	TP_SCL	PIN 29	BCM 5	软 I2C-3 触控时钟线 (复用原 SSD1306 总线)
14	TP_INT	NC	NC	✗ 触控中断: 采用纯轮询模式, 悬空不接
15	TP_RST	PIN 11	BCM 17	✓ 触控芯片复位 (必需的初始化信号)

📦 手动安装依赖与运行指南 (Manual Setup & Dependencies)

如果你不使用 Ansible, 需要在树莓派上直接运行 `3.5inch_Capacitive_Touch_LCD.py Demo`, 请遵循以下环境配置步骤 (特别针对基于 Debian 12 Bookworm 的最新 Raspberry Pi OS) :

1. 开启 SPI 和 I2C 接口

树莓派默认关闭这些硬件接口, 会导致运行报错 `No such file or directory`。运行前必须开启:

```
sudo raspi-config
# 选择 3 Interface Options -> I4 SPI -> Yes
# 选择 3 Interface Options -> I5 I2C -> Yes
sudo reboot
```


2. 安装 Python 系统级依赖

由于最新树莓派系统引入了 PEP 668 保护机制，禁止 `pip` 全局安装。同时，新系统内核弃用了旧的 `sysfs` GPIO 接口。因此我们必须使用 `apt` 安装，并使用 `rpi-lgpio` 替代旧的 `RPi.GPIO`：

```
# 1. 修复可能损坏的包状态（例如因为 32位 库导致的未满足依赖）
sudo apt --fix-broken install -y

# 2. 安装所有必需的 Python 第三方依赖
sudo apt install -y python3-rpi-lgpio python3-gpiozero python3-spidev python3-
```

3. 运行 Demo

赋予执行权限并使用系统自带的 `python3` 运行：

```
chmod +x 3.5inch_Capacitive_Touch_LCD.py
./3.5inch_Capacitive_Touch_LCD.py
```

为适配 RPi 4B/Bookworm 所做的核心修改

为了让微雪官方的 Demo 能够在最新的树莓派环境下稳定运行，我们在本项目中对原架构进行了以下关键性修改：

- 触控中断 (TP_INT) 改造为纯轮询模式:** 官方设计中，触控芯片 FT6336U 会通过 `TP_INT` 引脚发送硬件中断。为了避免与其它音频/硬件中断冲突并节省 GPIO 资源，我们彻底悬空了 `TP_INT` 引脚。在 Python 代码中，改用**纯轮询模式 (Polling)**（不断调用 `touch.get_touch_xy()` 结合较短的 `time.sleep(0.02)`）来获取触控坐标。
- 解决底层 GPIO 库冲突:** 将依赖中的 `RPi.GPIO` 替换为新系统官方推荐的兼容包 `rpi-lgpio`。这完美解决了由于内核升级导致的 `Conflicts: python3-rpi.gpio` 报错问题。得益于其 API 完全兼容，代码中依旧保留 `import RPi.GPIO` 即可无缝运行。
- Python 环境的 Shebang 规范化:** 将主程序首行的 `#!/usr/bin/python` 修改为了跨平台兼容性更好的 `#!/usr/bin/env python3`，确保系统能自动用正确的 Python3 解释器启动脚本。

自动化部署流程 (Ansible)

本项目的所有底层依赖、服务注册、以及复杂的 `dtoverlay` (软 I2C 开启、SPI 提速等) 均由 Ansible 一键接管。

1. 前置要求

- 目标树莓派已安装 **Raspberry Pi OS Bookworm (64-bit) Lite** 无头版本。
- 你的本地控制端电脑 (Mac/Linux) 已安装 `ansible`。
- 树莓派已接入网络，且配置了 SSH 免密登录。具体配置步骤如下：

1. 测试使用密码 SSH 登录到树莓派：

```
ssh <你的用户名>@<你的树莓派IP>
```

2. 在本地电脑生成 SSH 密钥（如果尚未生成）：

```
ssh-keygen -t rsa -b 4096
```

3. 将本地公钥复制到树莓派（使用 `ssh-copy-id` 或 `scp`）：

```
# 推荐使用 ssh-copy-id 一键配置：
```

```
ssh-copy-id <你的用户名>@<你的树莓派IP>
```

```
# 或者使用 scp 手动复制并配置：
```

```
scp ~/.ssh/id_rsa.pub <你的用户名>@<你的树莓派IP>:~/
```

```
ssh <你的用户名>@<你的树莓派IP> "mkdir -p ~/.ssh && chmod 700 ~/.ssh &&
```

4. 再次执行 `ssh <你的用户名>@<你的树莓派IP>`，确认能够免密直接登录。

2. 快速开始

在你的本地电脑终端执行：

```
# 1. 克隆项目
```

```
git clone [https://github.com/your-username/Ansible_RPI_TouchPlayer.git](http  
ed Ansible_RPI_TouchPlayer/ansible
```

```
# 2. 复制并编辑 hosts 文件，填入你的树莓派 IP
cp hosts.ini.example hosts.ini
nano hosts.ini

# 3. 一键执行自动化部署
ansible-playbook -i hosts.ini site.yml
```

(注：部署过程中若修改了底层的硬件 boot 配置，Ansible 会自动重启树莓派，属于正常现象。)

🔧 开发与调试

如果需要调整屏幕 UI 布局、字体大小或触控的"隐形热区"碰撞边界，只需修改 `scripts/` 目录下的 Python 核心文件，无需重新跑整个 Playbook。

你可以通过以下命令在树莓派上实时查看 UI 与触控服务的日志：

```
sudo journalctl -u touch_gui.service -f
```

🤝 贡献与许可

- 许可证：MIT License - 欢迎自由改造、分发。
- Made with ❤️ for the Maker Community.